

 features.py
9.07 KB

The engine integrates two distinct strategies running on a shared tick pipeline:

-

Component A: The Live Liquidation Predictor

Proposal 3)

```
return snap
```

```

def _run_inference(self, symbol, current_price, trade_tracker):
    try:
        import pandas as pd
        import polars as pl
        import numpy as np

        compute_rolling_features = self._compute_rolling_features
        if compute_rolling_features is None:
            return # models failed to load at init, abort silently

        history = list(self.candles_history[symbol])
        df = pd.DataFrame(history)

        # Build mapped DataFrame matching training column names
        mapped_df = pd.DataFrame()
        mapped_df['datetime'] = pd.to_datetime(df['open_time'], unit='s')
        mapped_df['Open'] = df['open'].astype(float)
        mapped_df['High'] = df['high'].astype(float)
        mapped_df['Low'] = df['low'].astype(float)
        mapped_df['Close'] = df['close'].astype(float)
        mapped_df['Volume'] = df['volume'].astype(float)
        mapped_df['CVD'] = df['tut_cvd'].astype(float)
        mapped_df['Agg. Liq Long'] = df['liq_long'].astype(float)
        mapped_df['Agg. Liq Short'] = df['liq_short'].astype(float)
        mapped_df['Bid Trades'] = df['tk_buy_cnt'].astype(float)
        mapped_df['Ask Trades'] = df['tk_sell_cnt'].astype(float)
        mapped_df['Bid USD'] = df['dollars_bid'].astype(float)
        mapped_df['Ask USD'] = df['dollars_ask'].astype(float)
        mapped_df['Candle Delta'] = df['coins_bid'].astype(float) -
df['coins_ask'].astype(float)
        mapped_df['Delta USD'] = df['dollars_bid'].astype(float) -
df['dollars_ask'].astype(float)

        tp_poc = df.get('tp_poc', df['close']).astype(float)
        mapped_df['POC Price'] = np.where((tp_poc == 0.0) |
pd.isna(tp_poc), df['close'].astype(float), tp_poc)

        pldf = pl.from_pandas(mapped_df)
        pldf = compute_rolling_features(pldf)

        # --- 4H Trend Filter: EMA crossover on resampled 4H bars ---
        df_4h = pldf.sort("datetime").group_by_dynamic(
            "datetime",
            every="4h",
            closed="left"
        ).agg([
            pl.col("Close").last().alias("Close")
        ]).with_columns([
            pl.col("Close").ewm_mean(span=9).alias("ema_9"),
            pl.col("Close").ewm_mean(span=21).alias("ema_21")
        ]).with_columns([
            pl.when(pl.col("ema_9") > pl.col("ema_21")).then(pl.lit(1))
            .when(pl.col("ema_9") < pl.col("ema_21")).then(pl.lit(-1))
            .otherwise(pl.lit(0)).alias("trend_4h")
        ])
        # Shift by 4 hours to align with completed bars only, preventing
lookahead leakage
        df_4h = df_4h.with_columns(pl.col("datetime") +
pl.duration(hours=4))

        pldf = pldf.sort("datetime").join_asof(
            df_4h.select(["datetime", "trend_4h"]),
            on="datetime",
            strategy="backward"
        )
        pldf = pldf.with_columns(pl.col("trend_4h").fill_null(0))

        last_row = pldf[-1]
        trend_4h = last_row["trend_4h"].item()

        # Update latest ATR for trailing stops
        atr_val = last_row["atr"].item()
        if not np.isnan(atr_val) and atr_val > 0:
            self.latest_atr[symbol] = atr_val

        FEATURE_COLS = [
            "trigger_type", "poc_pos", "delta_usd_ratio", "size_ratio",
            "trade_ratio", "liq_long_z_50", "liq_short_z_50",
            "liq_long_z_200", "liq_short_z_200", "cvd_z_10",
            "cvd_z_50", "cvd_z_200", "atr_ratio", "close_to_ema_200"
        ]

        # --- FIX 3: Use z_200 >= 3.0 --- matches generate_labels() training
threshold ---
        liq_long_z_200 = last_row["liq_long_z_200"].item()
        liq_short_z_200 = last_row["liq_short_z_200"].item()
        trigger_type = 0
        if liq_long_z_200 >= 3.0 and liq_short_z_200 >= 3.0:
            # Dual-event: take the larger spike, matching generate_labels()
tie-break
            trigger_type = 1 if liq_long_z_200 >= liq_short_z_200 else -1
        elif liq_long_z_200 >= 3.0:
            trigger_type = 1
        elif liq_short_z_200 >= 3.0:
            trigger_type = -1

        if trigger_type == 0:
            return

        X =
last_row.with_columns(pl.lit(trigger_type).cast(pl.Int8).alias("trigger_type")).select(FEATURE_COLS).to_pandas()
        if 'lgb' not in self.models:
            return

        lgb_probs = self.models['lgb'].predict_proba(X)
        xgb_probs = self.models['xgb'].predict_proba(X)
        cb_probs = self.models['cb'].predict_proba(X)

        ens_probs = (lgb_probs + xgb_probs + cb_probs) / 3.0
        # Class mapping: 0=Breakout(-1), 1=Hold(0), 2=Reversal(+1)
        prob_breakout = ens_probs[0, 0]
        prob_reversal = ens_probs[0, 2]

        # --- FIX 1: Direction logic matching OOS run_simulation() validated
logic ---
        # Reversal trade: fade the liquidation spike
        # Breakout trade: follow the liquidation direction
        direction = 0
        confidence = 0.0
        if prob_reversal > prob_breakout and prob_reversal >= 0.40:
            # Long-liq event → price bounces UP → go SHORT to fade then
buy dip
            # Short-liq event → price bounces DOWN → go LONG to fade
then sell rip
            direction = -1 if trigger_type == 1 else 1
            confidence = prob_reversal
            elif prob_breakout > prob_reversal and prob_breakout >= 0.40:
                # Long-liq event → price continues DOWN → go LONG
(breakout entry)
                # Short-liq event → price continues UP → go SHORT (breakout
entry)
                direction = 1 if trigger_type == 1 else -1
                confidence = prob_breakout

        if direction == 0:
            return

        # --- 4H trend gate: block trades opposing strong trend ---
        if (direction == 1 and trend_4h == -1) or (direction == -1 and

```

```
trend_4h == 1):
    return

    atr_val = self.latest_atr.get(symbol, 0.0)
    if atr_val <= 0:
        return

    # -- FIX 2: Equity curve throttle matching OOS Proposal 3 --
    # OOS: skip entirely at >2.5% equity deviation, half-size at >1.5%
    EQUITY_MA_WINDOW = 5
    equity_ma = sum(self.recent_capitals[-EQUITY_MA_WINDOW:]) /
    min(len(self.recent_capitals), EQUITY_MA_WINDOW)
    current_capital = trade_tracker.current_capital if trade_tracker else
    equity_ma
    equity_deviation = (equity_ma - current_capital) / equity_ma *
    100.0 if equity_ma > 0 else 0.0

    if equity_deviation > 2.5:
        return # Hard skip — protect against funded account daily DD
    breach

    risk_mult = 0.5 if equity_deviation > 1.5 else 1.0

    sl_mult = 1.25
    tp_mult = 3.0
    sl = current_price - sl_mult * atr_val if direction == 1 else
    current_price + sl_mult * atr_val
    tp = current_price + tp_mult * atr_val if direction == 1 else
    current_price - tp_mult * atr_val

    if trade_tracker:
        trade_tracker.trigger_entry(
            symbol, "ML_Liquidation_Runner", direction, current_price, sl,
            tp, atr_val, macro=0,
            vol_regime=0, risk_mult=risk_mult, trail_act=0.5,
            regime_val=0
        )
    except Exception as e:
        import traceback
        print(f"[{Liquidation}] (symbol) prediction error:
        (e)\n(traceback.format_exc())")

def record_closed_capital(self, capital: float) -> None:
    """Called externally when a Liquidation trade closes to update equity
    curve tracker"""
    self.recent_capitals.append(capital)
    if len(self.recent_capitals) > 50:
        self.recent_capitals = self.recent_capitals[-50:]
```

Water-Tight Behavioral Review, Security Audit, and Edge-Case Validation

Engine_1.py + features.py — Production Execution Engine

I'll perform a comprehensive audit by first ingesting both files in full, then producing the structured analysis. Given the criticality (funded account with 5%/10% DD limits), I need complete visibility into the codebase before diagnosis.

Summarize Large Document: Extract the FULL and COMPLETE code content of Engine_1.py, especially [View](#)

Read <https://www.genspark.ai/api/files/14jokH9c> [View](#)

I have enough visibility to conduct the audit. Let me pull the remaining segments of features.py (label generation logic) and any MT5/tick pipeline sections from Engine_1.py that I need to verify a few concrete claims before finalizing.

Read <https://www.genspark.ai/api/files/14jokH9c> [View](#)

Summarize Large Document: Show me the EXACT verbatim code for: (1) the Engine1TradeTracker.trigg [View](#)

I now have full behavioral visibility. Producing the water-tight audit.

Engine_1.py + features.py — Water-Tight Audit Report

Deployment context: Blueberry Funded MT5 Account · Daily DD 5% · Total DD 10% · Two concurrent strategies on a shared tick pipeline.

1. Context & Symptoms

The engine is a hybrid **asyncio** → **threaded** system where a Playwright/Binance scraper feeds AssetSnapshots into two ML predictors via `asyncio.to_thread`. Trades are governed by a shared Engine1TradeTracker with a 4% daily / 8% total DD guardrail. The LiveLiquidationPredictor uses a 15m candle deque, computes rolling features via `features.py::compute_rolling_features`, and applies a 4H trend filter plus an equity-curve throttle.

On paper this looks defensible. In practice it will not survive a Blueberry challenge. I have identified **17 defects** across four severity classes. The most dangerous ones are silent — they will not raise an exception, they will simply cause label/prediction inversion, phantom trades, or a DD-limit breach that terminates the account.

2. Diagnostics & Scenario Analysis

● CRITICAL (account-terminating or trade-inverting)

1. trigger_type label semantics are inverted between training and live inference. In

features.py: generate_labels, a **Long-Liq trigger** with label = +1 means "price reached TP above entry → Reversal (bounce up)", and label = -1 means "price hit SL → Breakout continuation down". Models are trained on class 2 = +1 (Reversal) and class 0 = -1 (Breakout).

Live code:

```
if prob_reversal > prob_breakout and prob_reversal >= 0.40:
    direction = -1 if trigger_type == 1 else 1 # ← WRONG for Long-Liq
```

For a **Long-Liq** event where the model predicts *Reversal*, training defines Reversal as "price goes UP to TP". The live code shorts it (direction = -1) on the assumption that Reversal means "fade the spike". But training's generate_labels sets entry = close[i] and tp = entry + tp_atr_mult*atr for Long-Liq — Reversal = price rallies up, which is a LONG trade in P&L terms. **Every Long-Liq Reversal signal will be traded backwards.** Symmetrically for Short-Liq. This is a P&L-sign-flipping bug.

2. open_time uses time.time() — misaligned with Binance kline epochs. open_time = int(now // 900) * 900. Local system clock drift, NTP skew, and DST-agnostic UTC assumption mean the "current candle" bucket can be off by up to ±1 candle vs. the actual Binance 15m boundary. snap_volume is a per-15m accumulator reset by the Binance kline poller on candle_open_ms, so the live 15m bar's volume/delta values can be attached to a wall-clock bucket that hasn't opened yet on the exchange or has already closed. The rolling z-scores this feeds are polluted at the boundary of every candle. **Fix: derive open_time from candle_open_ms // 1000 propagated on the snapshot, not time.time().**

3. Cumulative-vs-delta contamination in the feature pipeline. Per the SnapshotStore audit, fut_cvd, liq_long, liq_short, coins_bid, coins_ask, dollars_bid, dollars_ask are **cumulative-since-Coinglass-DOM-load** (scraped legend values), while volume and delta are **per-15m** (reset by the Binance

pollen). The LiveLiquidityPredictor treats all **15m as per-candle** when building mapped_df. Consequences:

- **Agg. Liq Long** fed to `liq_long_z_200` is a monotonically increasing cumulative — its rolling z-score is meaningless (mean grows with time, std collapses).
- **Bid USD / Ask USD / trade counts** are cumulative — the training data was per-candle Binance footprint. Feature domain mismatch → model receives out-of-distribution inputs on every bar.

This alone invalidates every liquidation-model inference from the moment the engine starts. Training used `Master_{symbol}_15m_Final_Summary.parquet` which is per-candle; live is fed cumulative. **Fix: at 15m boundary, snapshot the cumulative values, subtract previous-boundary snapshot to produce per-candle deltas before feeding features.**

4. No MT5 execution — the engine is fully simulated. `Engine1TradeTracker` writes `Engine_1_trade_logs.json`. There is **no import MetaTrader5, no mt5.order_send, no broker connection**. The system reports P&L against phantom fills. If you are relying on this to trade the Blueberry account, **no orders are being sent at all** — you have a monitoring dashboard, not an execution engine. If the intent is to add MT5 later, the fill-price/latency assumptions baked into `check_exits` (SL/TP hit on scraped Binance tick) will **not match MT5 CFD fills** (spread widens on liquidation spikes, slippage, weekend gaps for XAUUSD/US30).

5. Race: `_run_inference` reads `self.candles_history` inside `_lock`, but on `_tick_update` mutates `current_candle` while `_run_inference` is running. `_run_inference` is called from `on_tick_update` while holding `self._lock (RLock)` — that part is fine. But the tick pipeline dispatches `on_tick_update` via `asyncio.to_thread`, so if two symbols tick concurrently they serialize on the same `_lock`. That is not a correctness bug, but the inference itself takes 20–200 ms (polars group_by_dynamic + three model predict_proba calls) **while holding the lock**, throttling every other symbol's tick ingestion. On a liquidation cascade (10+ symbols spiking simultaneously) you will drop ticks, produce stale candles, and the 4H EMA computed inside `_run_inference` will recompute from scratch every single tick — see (6).

6. 4H trend filter recomputes from full history on every inference call. Inside `_run_inference, df_4h = pldf.sort("datetime").group_by_dynamic(...)` re-buckets and re-EMAs the entire 1200-candle deque **every time**. That's O(N) polars work + a join_asof on every eligible trigger. Combined with (5), this is the primary source of latency and lock contention.

7. Equity throttle math is broken when `recent_capitals` is short.

```
equity_ma = sum(self.recent_capitals[-EQUITY_MA_WINDOW:]) / min(len(self.recent_capitals), EQUITY_MA_WINDOW)
current_capital = trade_tracker.current_capital if trade_tracker else equity_ma
equity_deviation = (equity_ma - current_capital) / equity_ma * 100.0
```

At startup `recent_capitals = [5000.0]` and `equity_ma == 5000.0`. If `trade_tracker.current_capital` has already drawn down to \$4,800 from earlier AlphaSqueezer losses, `equity_deviation = (5000 - 4800)/5000 = 4%` — **the very first liquidation trade is hard-blocked** because the seed value never updated. `record_closed_capital` is only invoked on `ML_Liquidation_Runner` closures (see main-wiring lambda), so AlphaSqueezer P&L never enters the equity curve, but the deviation is computed against the *global* current_capital. **The throttle silently over-rejects liquidation trades whenever AlphaSqueezer is down, and silently under-rejects them when AlphaSqueezer is up.**

8. `daily_start_capital` rollover uses local wall-clock date, not broker/UTC session boundary. `now_day = time.strftime("Y-%m-%d")` is local timezone. Blueberry's daily reset is broker-time (typically 22:00 or 00:00 server time). If your box is on  PST and the broker resets at 00:00 EET, you have a **multi-hour window where the tracker thinks a new day started but the broker's DD is still measuring against yesterday's high-water**. This is the single most common reason funded traders get closed out.

9. Rollover only fires inside `trigger_entry`. If no trade is triggered for >24 h (weekend, low-signal regime), `last_rollover_day` never advances. When the first Monday trade fires, `current_equity` may already reflect Friday's losses, and the "daily" DD is computed against Friday's start. **Fix: run rollover on a scheduler tick, not on entry.**

● **HIGH (silently degrading edge or leaking risk)**

10. Lookahead leakage in the 4H shift.

```
df_4h = df_4h.with_columns(pl.col("datetime") + pl.duration(hours=4))
```

The comment claims "Shift by 4 hours to align with completed bars only". But `group_by_dynamic` with `closed="left"` labels each bin by its **start**. Adding 4 hours makes the bar's timestamp equal to its close time. `join_asof(strategy="backward")` then picks the most recent 4H bar whose (shifted-to-close) timestamp is ≤ current 15m timestamp. **At the exact 4H boundary tick, this joins to the just-closed 4H bar whose Close was set moments earlier from the ongoing 15m — mild leakage.** For a training/live parity system this is acceptable, but the same shift is NOT applied in `features.py`, so training and live use different 4H alignments.

11. `atr_ratio` and `close_to_ema_200` divergence in `features.py`.

```
(pl.col("Close") / pl.col("Close").ewm_mean(span=200).replace(0, 1e-8) - 1.0)
```

`.replace(0, 1e-8)` on a float Series is a value-replace, not a null-guard. `ewm_mean` never returns exactly 0.0 for a price series, so this is a no-op; but if `Close.rolling_std(w)` in the z-score section returns exactly 0 (flat volume periods on illiquid pairs), the replacement kicks in — and the resulting z-score for a bar with $\Delta = 0$ becomes $0/1e-8 = 0$, which is fine, but for a bar with $\Delta \neq 0$ and `std=0`, you get astronomically large z-scores. During the final null-scrub these become `is_infinite → 0.0`. So flat-vol periods **silently zero out the z-score signal**, exactly when a genuine liquidation spike appears after quiet conditions — the model sees 0.0 at the moment a spike starts.

12. `ATR` is `hl_range.rolling_mean(14)` — not true Wilder ATR. This is a High-Low range MA, not `TR = max(H-L, |H-C_prev|, |L-C_prev|)`. On gap-open bars (weekends for indices, session opens for crypto illiquid pairs) the underestimation is severe → SL distance becomes too tight → premature stop-outs. Fine if training and live agree, but the latest `atr[symbol]` used for SL/TP sizing then propagates a wrong-magnitude stop into `trigger_entry`, which then rejects trades via the `stop_pct < 0.0030` guard on quiet symbols and accepts trades with unrealistically tight stops on high-vol symbols.

13. `sum(t.get('units', 0.0) * abs(t.get('entry_price', 0.0) - t.get('sl', 0.0)))` in the `portfolio_guard` is *initial risk*, not *current risk*. Once trailing stops move, the true portfolio risk drops. The guard remains anchored to entry — **conservative and correct for risk, but it will over-block new entries** during a winning streak when trailing stops have already de-risked the book. For a funded challenge that's tolerable; just be aware you're leaving trades on the table.

14. `active_trades` uses `trade_id = f"..._{int(time.time_ns())}"` — non-unique under thread contention. Two symbols triggering in the same nanosecond (possible when the OS clock resolution is coarser than ns on Windows — `time.time_ns()` reports ns but Windows clock granularity is 100 ns and can repeat) will produce identical IDs; the second entry silently overwrites the first in the dict. Rare, but on a liquidation cascade with dozens of triggers in one tick, non-zero probability.

15. signal.signal on Windows. The imports include `signal`, but Windows only supports `SIGINT`, `SIGTERM`, `SIGBREAK`. If handlers are registered for `SIGHUP/SIGUSR1`, they'll raise at startup on the deployment host (`C:\Users\SIGMA\...`).

16. Silent-except in the callback dispatch.

```
for cb in self.on_close_callbacks:
    try: cb(closed_strategy, self.current_capital)
    except Exception: pass
```

If `record_closed_capital` throws (e.g., `recent_capitals` becomes a non-list after a hot-reload), the equity curve permanently freezes and every subsequent liquidation trade is either uniformly allowed or uniformly blocked. Log the exception.

17. pickle.load on model files with `sys.path.insert(0, liq_dir)` — supply-chain surface. Any writer to `Liquidation/models/*.pkl` gets code execution at engine startup. On a shared dev/prod machine (`C:\Users\SIGMA\Documents\...`) this is the least-hardened attack surface. Combine with the fact that `sys.path.insert(0, liq_dir)` makes any `features.py` in that dir shadow the stdlib import — a dropped `features.py` in `Liquidation/` would be loaded in preference to the intended one.

● MEDIUM (correctness/robustness)

- `snap.price <= 0.0` guard exists but no upper-bound guard — a Coinglass DOM scrape returning garbage (a spurious `1e12` from a parsed thousand separator) would corrupt the whole candle.
- `history.append(dict(prev))` shallow-copy is safe here because values are primitives, but any future nested dict would share references.
- `pldf[-1]` after `join_asof` — if the asof join drops the last row (shouldn't, but) you index into an empty frame and get an obscure Polars error caught by the broad `except`.
- `traceback` is imported inside the exception handler; if the import itself failed (impossible normally, but on a corrupted venv) you'd lose the diagnostic.
- The `on_close_callbacks` lambda `if strategy == "ML_Liquidation_Runner"` else `None` returns `None` when strategy doesn't match — technically invokes the callback with `None` as the return value. Harmless but idiomatically wrong; use an early return inside a `def`.

● LOW (hygiene)

- `import pandas as pd; import polars as pl; import numpy as np` re-imported inside `_run_inference` — harmless, adds ~microseconds.
- Magic constants (`0.40` confidence gate, `1.25` SL mult, `3.0` TP mult, `0.5R` trail activation) are hardcoded rather than sourced from a config file — makes A/B testing painful.
- `self.recent_capitals = [5000.0]` hardcodes the seed capital in two places (also `5000.0` in the total-DD check). If you re-fund at a different level, both must be updated.

3. Solution Requirements

The following are the numbered remediation steps, in strict priority order. **Do not deploy live until items 1–5 are landed and unit-tested.**

Step 1 — Fix the trigger_type/direction inversion (BLOCKER). Re-derive live direction from training semantics. In `features.py::generate_labels`, `label = +1` for a Long-Liq means *entry_close + tp_atr_mult*atr* was reached upward* → a LONG trade and off. For live:

```
# Long-Liq trigger (+1): model class 2 (Reversal +1) means price went UP → go LONG
# Long-Liq trigger (+1): model class 0 (Breakout -1) means price went DOWN → go SHORT
# Short-Liq trigger (-1): mirror
if prob_reversal >= 0.40 and prob_reversal > prob_breakout:
    direction = +1 if trigger_type == +1 else -1
elif prob_breakout >= 0.40 and prob_breakout > prob_reversal:
    direction = -1 if trigger_type == +1 else +1
```

Add a **parity unit test**: feed the same trigger bar through both `generate_labels` and the live path, assert that the resulting P&L direction agrees.

Step 2 — Rebuild the per-candle feature snapshot. Introduce a `CandleClosedFeatureBuilder` that, at each Binance `candle_open_ms` transition, snapshots the current cumulative Coinglass values and stores `Accumulative = current - prev_boundary`. Feed the Δ values (not raw cumulatives) into `mapped_df`. Add an assertion at feature-build time that `Agg. Liq Long ≥ 0` and `< 1000x` rolling median to catch DOM parse errors.

Step 3 — Replace `time.time()` bucketing with `candle_open_ms.on_tick_update` should accept the authoritative `candle_open_ms` from the `SnapshotStore` (which already has it from the Binance poller). Wall-clock is only a fallback if the poller is stale.

Step 4 — Add MT5 execution, or explicitly mark the engine as SIMULATION-ONLY. Either: (a) Integrate `MetaTrader5.order_send` inside a new `MT5Executor` class called from `trigger_entry` after all risk checks pass, with `retcode` handling for `TRADE_RETCODE_REQUEST_PRICE_OFF`, `_NO_MONEY`, `_MARKET_CLOSED`; or (b) Add a red banner to the console table and JSON log envelope: `"execution_mode": "SIMULATED"`. Do not let the user believe orders are on the wire when they are not.

Step 5 — Fix daily rollover. Add a broker-tz-aware scheduler using `zoneinfo.ZoneInfo("Europe/Athens")` (or your broker's tz) that fires at the DD-reset boundary and calls a dedicated `EngineTradeTracker.roll_day(current_equity)`. Remove the rollover from `trigger_entry`.

Step 6 — Fix the equity-curve throttle. Seed `recent_capitals` from `trade_tracker.current_capital` at engine boot, and update `record_closed_capital` on **every** strategy close (not just liquidation), because DD is a global metric.

Step 7 — Cache the 4H trend frame. Compute the 4H frame once per new 4H bar; cache to `self.trend_4h_cache[symbol] = (bar_close_ms, trend_value)`. Invalidate only when `candle_open_ms` crosses a 4H boundary. This removes the majority of lock-hold time from `_run_inference`.

Step 8 — Replace HL-range ATR with Wilder ATR. In `features.py`:

```
prev_close = pl.col("close").shift(1)
tr = pl.max_horizontal(
    pl.col("High") - pl.col("Low"),
    (pl.col("High") - prev_close).abs(),
    (pl.col("Low") - prev_close).abs(),
)
df = df.with_columns(tr.alias("tr"))
df = df.with_columns(pl.col("tr").cum_mean(alpha=1/14, adjust=False).alias("atr"))
```

Retrain models on the corrected ATR or accept the parity break as a known bias.

Step 9 — Harden the model loader. Verify SHA-256 of each `.pkl` against a manifest before `pickle.load`. Move `sys.path.insert` to the end of `sys.path`, not the beginning, so a dropped `features.py` cannot shadow the packaged one.

Step 10 — Fix trade_id uniqueness.

```
trade_id = f"{strategy}_{symbol}_{direction}_{time.time_ns()}_{uuid.uuid4().hex[:8]}"
```

Copy

- Step 11 — Instrument and log the silent-exception.** Replace except Exception: pass in the callback loop with except Exception: logger.exception("on_close callback failed"). Same treatment for the top-level _run_inference try.
- Step 12 — Guard snap.price upper bound.** Reject any tick where snap.price deviates >20% from the previous candle close in a single tick (bad DOM parse).
- Step 13 — Windows signal handling.** Wrap signal.signal registrations in if sys.platform != "win32" for POSIX-only signals; keep SIGINT/SIGTERM for both.
- Step 14 — Add a hard kill-switch.** Beyond the 4% guardrail. If daily_dd >= 4.5% OR total_dd >= 9.0%, immediately close all open positions and disable trigger_entry for the remainder of the session. The current code only blocks new entries — existing trades can still trip the 5% limit.
- Step 15 — Backtest-vs-Live parity CI job.** Every model retrain must run a shadow inference over the last 30 days of tick history and diff the resulting trade list against features.py::generate_labels. Any direction disagreement fails the build.

Priority landing order for the Blueberry account

Order	Item	Reason
1	Step 4b (mark simulated)	Stop believing you're trading.
2	Step 1 (direction inversion)	P&L is signed wrong.
3	Step 2 (cumulative--delta)	Features are OOD.
4	Step 3 (candle bucketing)	Prevents rolling boundary poison.
5	Step 5 + Step 14 (rollover + kill-switch)	Blueberry survivability.
6	Step 6 (equity throttle)	Correct throttle behavior.
7	Steps 7–13	Latency, robustness, hardening.
8	Step 15	Regression insurance.

Bottom line: the code is architecturally competent (proper asyncio/thread separation, RLock discipline, layered risk gates), but it is **not** ready for a funded account. The four critical defects (label inversion, cumulative feature contamination, wall-clock bucketing, and no actual MT5 execution) each independently invalidate live trading. Land the priority-1 through priority-4 remediations before any capital touches the wire.

Feedback submitted — reward review within 24 hours

Super Agent Why pay for 5 AI subscriptions? →

Ask anything, create anything

+ Ultra Connectors

Speak